ECSE 2920: Design Methodology

Technical Specification and Description Report

Group 5

Table of Contents

Introduction **3**

GitHub Repository Setup **4**

Creating SSH Key *4*

Pushing Repository Changes to GitHub **5**

Python Environment Setup on the Raspberry Pi *7*

GitHub Integration on the Raspberry Pi *7*

Obtaining Voltages on Raspberry Pi *7*

DigiPot Setup and Rotary Encoder **8**

Accessing and Using a GPIO **9**

Software Logic and Library Implementation *10*

System I/O and GPIO Allocation **10**

Power Supply Simulation Check **11**

PCB File Review and "Blank Hat" Starting Point **13**

Introduction

Our group at the beginning of the semester was tasked with designing and building an Electronics Tool Bench, like the equipment found in the Circuit Labs in the Driftmier Engineering Center.

1. General Parts

Part	Comments	Quantity	Student Replace	Link
Dupont Wires	FF,FM,MM	120 (40 each)	Return	
Wire Jumper kit		1	Return	
Breadboard kit (Rexqualis)		1	Return	https://www.amazon.com/Breadboards-Solderless-Breadboard-Distribution-Connecting/dp/B07DL13RZH/ref=sr_1_16?crid=2GEGFKZKZRKEM&keywords=proto+board&qid=1682345037&sprefix=proto+board%2Caps%2C105&sr=8-16
Pi Hat Protector	2 fuses	1		
24V Power Supply		1	Yes	https://www.amazon.com/ALITOVE-100-240V-Adapter-Convertor-5-5x2-1mm/dp/B01GC6VS8I/ref=sr_1_3?adgrpid=134470329143772&hvadid=84044027575314&hvbmt=be&hvdev=c&hvlocphy=4587&hvmete=o&hvqmt=e&hvtargid=kwd-84044127904&hvpone=&hvpos=&hvnetw=o&hvrand=4587

GitHub Repository Setup

One group member must access the GitHub website to create a new Git repository as a place for all group members to archive all documentations and pictures. The same group member clicked the "+" dropdown menu, then clicked New repository, then named it "Group-5". All group members created individual directory hierarchies using their OS terminals (or command prompts) to store all the pulled deliverable repositories and documentations. This is done using the following commands:

```
$ mkdir [directory_name] - creates a new directory
$ cd [directory_name] - changes to the target directory
$ cd . - changes to the home directory
```

Because the repository creator is discouraged from being the only group member to submit progress for the project, the remaining group members must have remote access to the Group-5 repository to commit and push necessary changes.

The individual group members can clone the repository by executing this Git command:

```
$ git clone [SSH Address]
```

This way of cloning the Git repository using an SSH key is more secure than HTTPS because the data is encrypted by a key pair; a private key is handled by a group member to sign, and a public key is handled by GitHub but kept in the group member's Git account. However, to execute the command, all group members must create an individual private SSH key on their personal computer.

Creating SSH Key

Each group member generates an SSH key by opening the terminal and running this command:

```
$ ssh-keygen -t ed25519 -C "[student_email@uga.edu]"
```

When the terminal displays the following messages:

> Generating public/private ALGORITHM key pair.
> Enter a file in which to save the key (/Users/YOU/.ssh/id_ALGORITHM):

The ↵ Return key is pressed to save the key to the default file location:

Including the passphrase for the SSH key is optional.
> Enter passphrase (empty for no passphrase): [Type a passphrase]

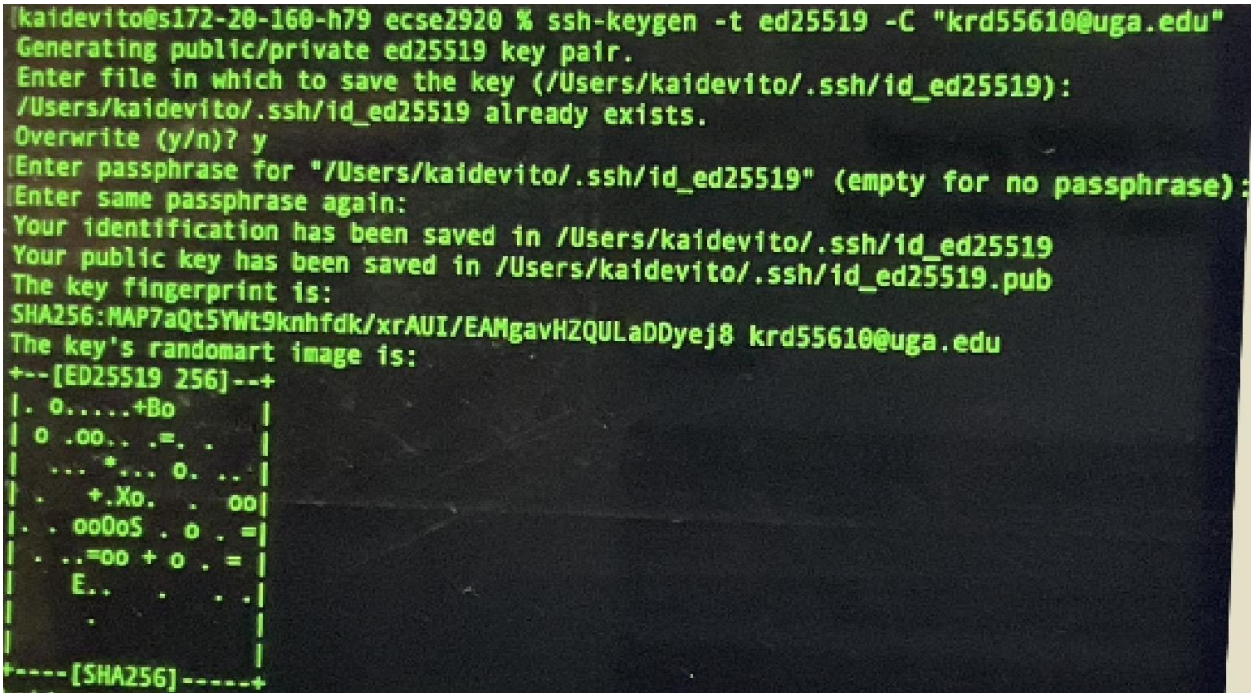

Figure 1. The image of the terminal showing the successfully created public SSH key.

Once the public key is saved into the .pub file, each group member copies and displays its contents like so:

```
$ pbcopy < ~/.ssh/id_ed25519.pub
$ pbpaste
```

The result from $ pbpaste is then pasted into the textbox under GitHub Profile Picture -> Settings -> SSH and GPG keys -> New SSH key.

Pushing Repository Changes to GitHub

All group members had to create a Python file that prints out their individual names using the print() function to then push to the repository. Before executing the commands, all group members reviewed the Git Workflow and its corresponding commands to traverse each member's saves from each member's personal computers to the Git repository.

There are four areas where saves are kept: the working repository on the computer, then the staging area for organizing the files for pushing to the Git repository, then the local repository storing a copy of the Git repository, and finally the remote (Git) repository itself.

This is the sequence of Git commands each member had to run to push repository changes:

$ git add [file_name] – moves a file from working repository to staging area
$ git add . - moves all saves from working repository to staging area
$ git commit -m "message" - moves saves from staging area to local repository
$ git push - moves saves from local repository to remote (Git) repository

$ git status - verify current states of working directory and staging are

Raspberry Pi Setup and Initial Configuration
After all group members had access to the shared GitHub repository, the group began setting up the Raspberry Pi 4 as the primary hardware platform for the project. The Raspberry Pi operating system was installed on one of the group member's computers and was configured to allow the device to function independently as a development and demonstration system. Once the system was running, all group members verified that the Raspberry Pi was operating correctly and ready for software setup.

During this process, the group ran into difficulties connecting the Raspberry Pi directly to the UGA WiFi network. Because the network requires authentication methods that are not easily supported by the Raspberry Pi, the device was unable to maintain a stable wireless connection. To resolve this issue, an Ethernet cable was used to create a bridge connection between the Raspberry Pi and one of our teammate's laptops. The computer was connected to the UGA WiFi network and shared its Internet connection with the Raspberry Pi through the Ethernet port. This approach allowed the Raspberry Pi to reliably access the Internet and continue the setup process without interruption.

Once the bridge connection was established, the group could update the system and prepare the Raspberry Pi for development use.

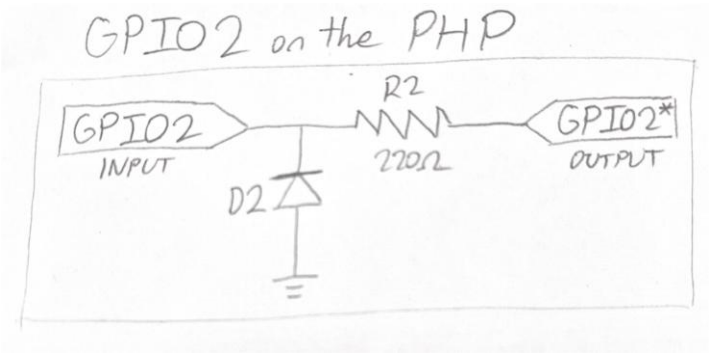

Figure 2. Shows a Technical Drawing of the GPIO2 port on the PHP (Pi Hat Protector) Zener Diode, 220 Ohm Resistor, GPIO2 output, GPIO2 input

When the Zener Diodes breakdown voltage is exceeded by its' polarity, the current flows in the opposite direction.

The group additionally made a Technical Drawing of one of the GPIO ports on the Raspberry PI to better get an idea of how they would be used in the future for necessary Inputs and Outputs.

Python Environment Setup on the Raspberry Pi
With reliable Internet access available, the group confirmed that Python was installed and functioning correctly on the Raspberry Pi. Since Python is the primary language used throughout the project, it was important to ensure that scripts could be written and executed without issues.

To keep project dependencies organized and separate from the system environment, the group created a Python virtual environment on the Raspberry Pi. This allowed the installation and management of project specific libraries without affecting other software on the device.

GitHub Integration on the Raspberry Pi
After confirming that Python was working correctly, the group set up Git on the Raspberry Pi so it could be used as part of our normal development workflow. The shared Group 5 repository was cloned onto the Raspberry Pi, which allowed files stored on GitHub to be accessed and run directly on the device.

To verify that this integration was working as expected, one of the group members ran the Hello_WorldPI.py file from the repository on the Raspberry Pi. Successfully executing this file confirmed that code pulled from GitHub could be run on the hardware without modification.

Obtaining Voltages on Raspberry Pi

When the group was completing Deliverable 3 and testing the needed components, there was an error in obtaining the proper 3.3V to build a simple LED/Resistor circuit.

The error was in attempting to obtain the voltage using the GPIO extender on the breadboard from the Pi Hat Protector. Researching the proper ribbon cable direction and orientation, we first figured that our ribbon cable seemed to not match the proper color orientation, so that was why we were not obtaining 3.3V through the GPIO extender. However, after further group discussion, we realized that the GPIO extender may be unable to supply 3.3V, and we instead used the 3.3V port on the Pi Hat Protector to obtain the needed 3.3V for our LED and Rotary encoder.

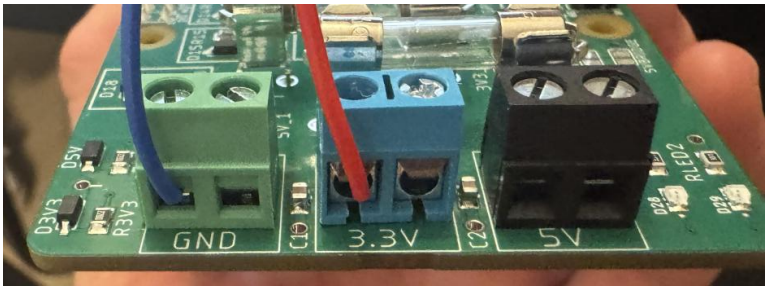

Figure 3. Shows the GND, 3.3V, 5V to obtain needed voltage since the GPIO extender wouldn't supply our voltages.

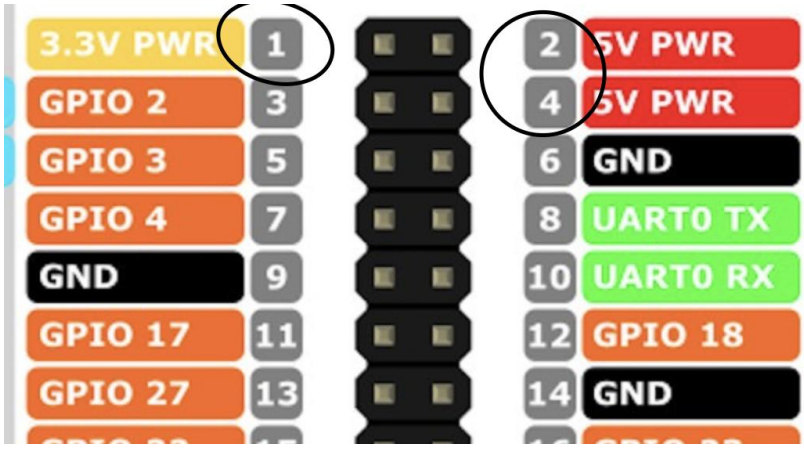

Figure 4. https://roboticsbackend.com/raspberry-pi-3-pins/
Shows the diagram of the Raspberry Pi Pins- noting the 5V PWR and 3.3V PWR pins.

Using female adapters on these pins can also obtain the needed 3.3V, 5V, GND, and access the GPIO ports.

DigiPot Setup and Rotary Encoder

The MCP4131 digital potentiometer communicates with the Raspberry Pi 4 using SPI (Serial Peripheral Interface). SPI is a synchronous serial protocol where the Raspberry Pi acts as the master and the potentiometer acts as the slave. The master provides the clock (SCK), selects the device using chip select (CS/CE), and sends command/data bytes on MOSI to set the wiper position.

SPI uses four main signal lines: MISO (master in), MOSI (master out), CS/CE (chip select), and SCK (clock). For this project, the important part is sending commands from the Pi to the potentiometer, so MOSI, SCK, and CS/CE are required. MISO is optional unless the program is reading data back.

The following connections between the MCP4131 and the Raspberry Pi 4 were made:

- CS (Chip Select Input; Pin 1) to SPI0 CE0 (GPIO8) (or CE1/GPIO7 if configured that way in code)
- SCK (Serial Clock; Pin 2) to SPI0 SCLK (GPIO11)
- SDI (Serial Data In; Pin 3) to SPI0 MOSI (GPIO10)
- SDO (Serial Data Out; Pin 3, optional) to SPI0 MISO (GPIO9) (only needed if reading back)
- VSS (Pin 4) to Ground
- P0A (Terminal A; Pin 5) to Ground (for our measurement setup)
- VDD (Pin 8) to 3.3V Power

https://www.geeksforgeeks.org/electronics-engineering/what-is-serial-peripheral-interface-spi/

The resistance values of the digital potentiometer are measured with a multimeter device, which connects the two leads of the potentiometer like so:

- P0W (Potentiometer Wiper Terminal; Pin 6 on MCP4131, Pin 9 on MCP4231) to Multimeter COM (Ground)
- P0B (Potentiometer Terminal B; Pin 7 on MCP4131, Pin 10 on MCP4231) to Multimeter Positive Input

The digital potentiometer has a rotary track of resistant material within most of its circumference sandwiches between Terminals A and B, and the wiper serves as a voltage divider to provide an output resistance. The 8th slide of this presentation shows a top-down view of the device with the roles terminals eminently shown.

Accessing and Using a GPIO

Each of the Raspberry Pi 4 GPIOs are accessible through functions of the Python *pigpio* library. Because of the OOP architecture of the library, all the library's functions are methods of the *pigpio* class and are called via this syntax: pigpio.[function_name()].

Datasheet: https://ww1.microchip.com/downloads/en/DeviceDoc/22060b.pdf

This is a list of its methods used to communicate with the GPIOs alongside their descriptions:

- pigpio.pi() – initializes the pigpio class
- spi_open(SPI_CHANNEL, SPI_SPEED, SPI_FLAGS) – opens the SPI handle (the number referencing the SPI device, the potentiometer), based on the desired SPI channel, speed of the SPI communication (bits per second), and flag settings
- spi_write(HANDLE, BYTES_TO_WRITE) – writes the data bytes provided in a list [] to the handle
- spi_close(HANDLE) – closes the SPI handle
- stop() – stops the Raspberry Pi connection and drops pigpio resources

pigpio library: https://abyz.me.uk/rpi/pigpio/python.html#spi_write

Both the MCP4131 and the MCP4231 are 7-bit digital potentiometers, which means that the range of resistance steps, or increments between Ohm (Ω) values, is [0, 128]. In the Python program potentiometer.py, which uses calls to the *pigpio* library, the constant MAX_STEPS is set to 128 = 2^7, offering 129 total steps from 0 to the constant MAX_OHMS value, which is 10,000.

To program the various values of resistance, the Python program opens the SPI handle, the number that represents the potentiometer, using a chosen SPI channel, bits per second, and flags settings. Then, the program attempts to start the cycle to approximate the resistance across Terminal A and the wiper by iterating through the necessary Ohm (Ω) values and the step values in [0, MAX_STEPS] in two modes. Each iteration has some delay measured in seconds.

First, an Ohm value is converted to a byte step value in the range [0, MAX_STEPS]. Before this, however, the program ensures that for all Ohm values, 0 ≤ Ohm value ≤ MAX_OHMS. If the Ohm value is less than 0, then it is reassigned to 0, but if the Ohm value is greater than MAX_OHMS, then it is reassigned to MAX_OHMS. Then, the corresponding step value is found using this formula:

```
step_value = int((ohm_value / MAX_OHMS) * MAX_STEPS)
```

Second, the program sets the potentiometer steps by writing the data bytes to the potentiometer's SPI handle. If the supplied step value is in the range [0, MAX_STEPS], the program writes the command bytes to the potentiometer's SPI handle and prints the expected approximate resistance to verify with the multimeter. This approximate Ohm value is found by performing the conversion from byte steps to Ohms using the formula. But, if the supplied step value is outside this range, then the program warns the user that the supplied step value is invalid. To reiterate, the first mode of the cycle requires converting Ohms to steps in each iteration, while the second mode does not require converting units.

Software Logic and Library Implementation

We used the Python pigpio library to access GPIO and SPI features on the Raspberry Pi. Before running the program, the pigpio daemon must be active so the script can connect to the Pi's GPIO resources. In code, the connection is created using pi = pigpio.pi(), then SPI is opened using spi_open(...) and command bytes are sent with spi_write(...).

For inputs (such as the rotary encoder switch or encoder channels), internal pull-up or pull-down resistors are used so the input pin does not float when the switch is open. This prevents random state changes caused by noise and makes the input behavior consistent during testing.

System I/O and GPIO Allocation

To keep the hardware organized and prevent pin conflicts, a GPIO reference was created that lists the Raspberry Pi GPIO names, their functional role in the project, their default power-up state, and the state used during normal operation. Each signal also shows whether a pull-up or pull-down is used, along with the source connection for inputs and the drive destination for outputs. This makes it easier to understand what each pin is doing without tracing wires on the breadboard.

RP4 GPIO Diagram					
GPIO Names	Power Up Default State	State Used	Pull Up or Pull Down	Source Connection (for inputs)	Drive Destination (for outputs)
GPIO17	INPUT	INPUT	Internal pull-up	KY-040 Rotary Encoder SW	N/A
GPIO18	INPUT	INPUT	Internal pull-up	LED	N/A
GPIO9	INPUT	SPI0 MISO	N/A	MCP4131/MCP4231 SDO	N/A
GPIO11	INPUT	SPI0 SCLK	N/A	N/A	MCP4131/MCP4231 SCLK
GPIO10	INPUT	SPI0 MOSI	N/A	N/A	MCP4131/MCP4231 SDI
GPIO8	INPUT	SPI0 CS0	N/A	N/A	MCP4131/MCP4231 CS
GPIO2	INPUT	OUTPUT	N/A	N/A	LCD SDA
GPIO3	INPUT	OUTPUT	N/A	N/A	LCD SCL

Figure 5. This figure documents which Raspberry Pi GPIO pins are assigned to each project function and shows signal direction (input into the Pi vs output from the Pi). Including default power-up state and pull configuration helps avoid floating inputs and unexpected behavior during startup. This figure is important because it prevents pin conflicts and speeds up debugging when the system is expanded.

To avoid power mistakes, we also documented how power is distributed across the system. The power reference lists each device that requires power and shows which voltage rail it uses. This makes it easy to confirm correct voltage selection (3.3V vs 5V) and shared ground.

RP4 Power Diagram				
Device/Load	Power Pins	Voltage Required	Current Required	Power Source
Raspberry Pi 4	USB-C Input	5 V	N/A	USB-C Pi Power Supply
Rotary Encoder (KY-040)	VCC, GND	3.3 V	N/A	Pi 3.3V Rail + Pi GND
MCP4131/MCP4231 (Digipot)	VDD/ VSS (GND)	3.3 V	N/A	Pi 3.3V Rail + Pi GND
LCD	VCC, GND	5 V	N/A	Pi 5V Rail + Pi GND

Figure 6. This figure shows how power is routed to each device and what voltage rail is sued. This is important because using the wrong rail can damage hardware, and missing a shared ground can cause unstable or incorrect readings. Having this documented makes hardware bring-up and troubleshooting much faster.

Power Supply Simulation Check

We simulated a ± 12V DC supply derived from a 24V source to check that both output rails are stable with no load and under load. This let us verify expected beahviro before building it physically.

During testing, we found that a single 240Ω resistor on each rail produced approximately 50 mA, but it exceeded its power rating over time due to power dissipation. To fix that, we used four 60Ω resistors on each rail so the power is spread across multiple ¼W resistors. This keeps the load behavior similar while reducing overheating risk.

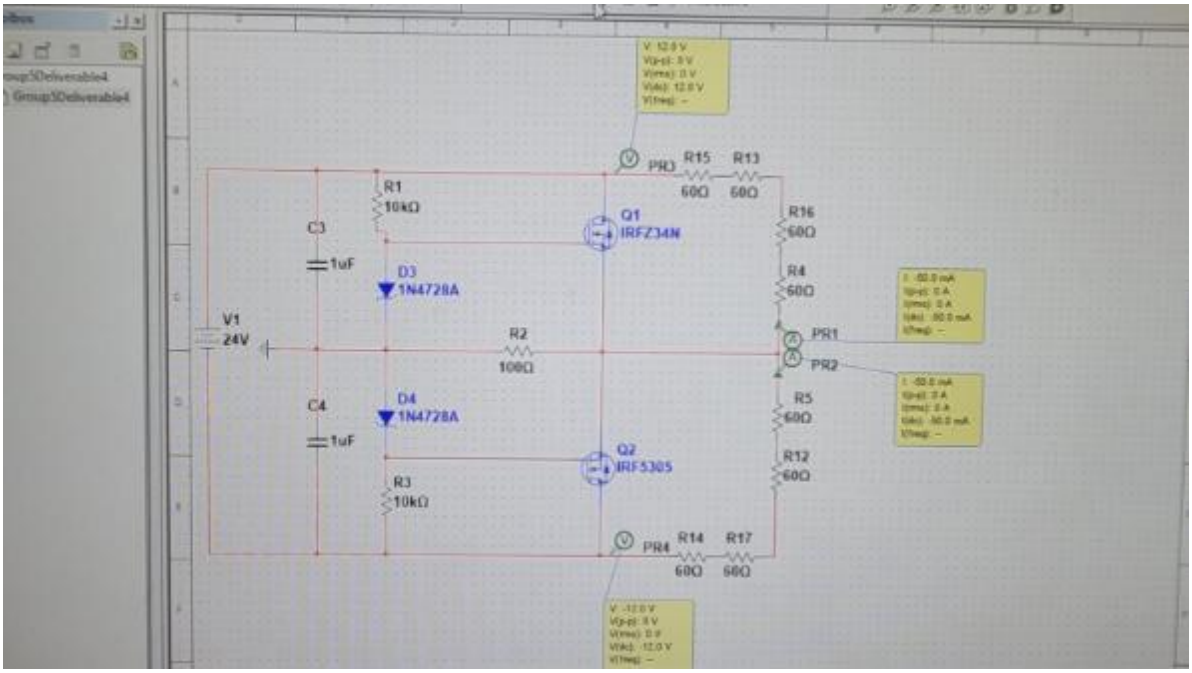

Figure 7. This figure shows the simulated power supply operation without resistive loading. It verifies that the circuit produces the expected positive and negative reference outputs from the 24V input. This matters because it confirms the supply works in the simplest case before checking performance under load.

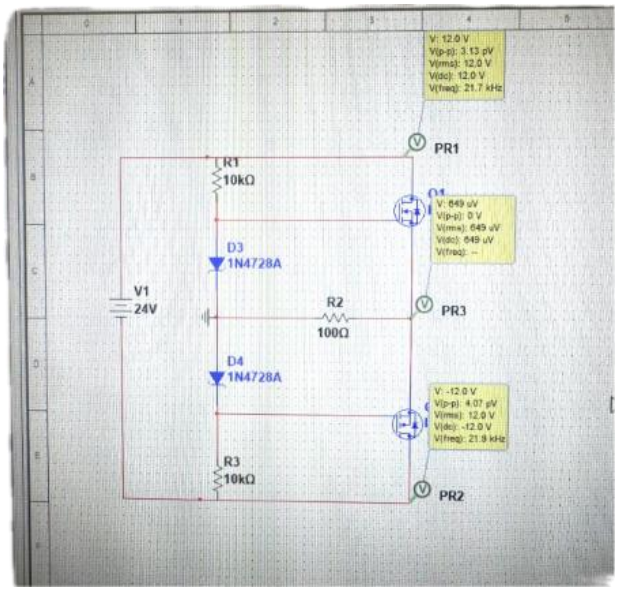

Figure 8. This figure shows the supply with resistive loads added to simulate current draw and evaluate stability. It is important because it demonstrates whether the output rails stay near ±12V under realistic conditions and helps confirm components are not pushed past safe operating limits.

PCB File Review and “Blank Hat” Starting Point

To prepare for PCB work, we opened the provided Pi Hat Protector (.brd and .sch) files and reviewed the routing needed to map the Raspberry Pi GPIO header. This helped us understand routing constraints and why multiple PCB layers are used. Multiple layers mainly provide routing space and cleaner signal/power paths.

We also reviewed the protection components. Zener diodes are oriented opposite of regular diodes so they can clamp voltage when reverse-biased, helping protect GPIO pins from spikes or incorrect wiring.

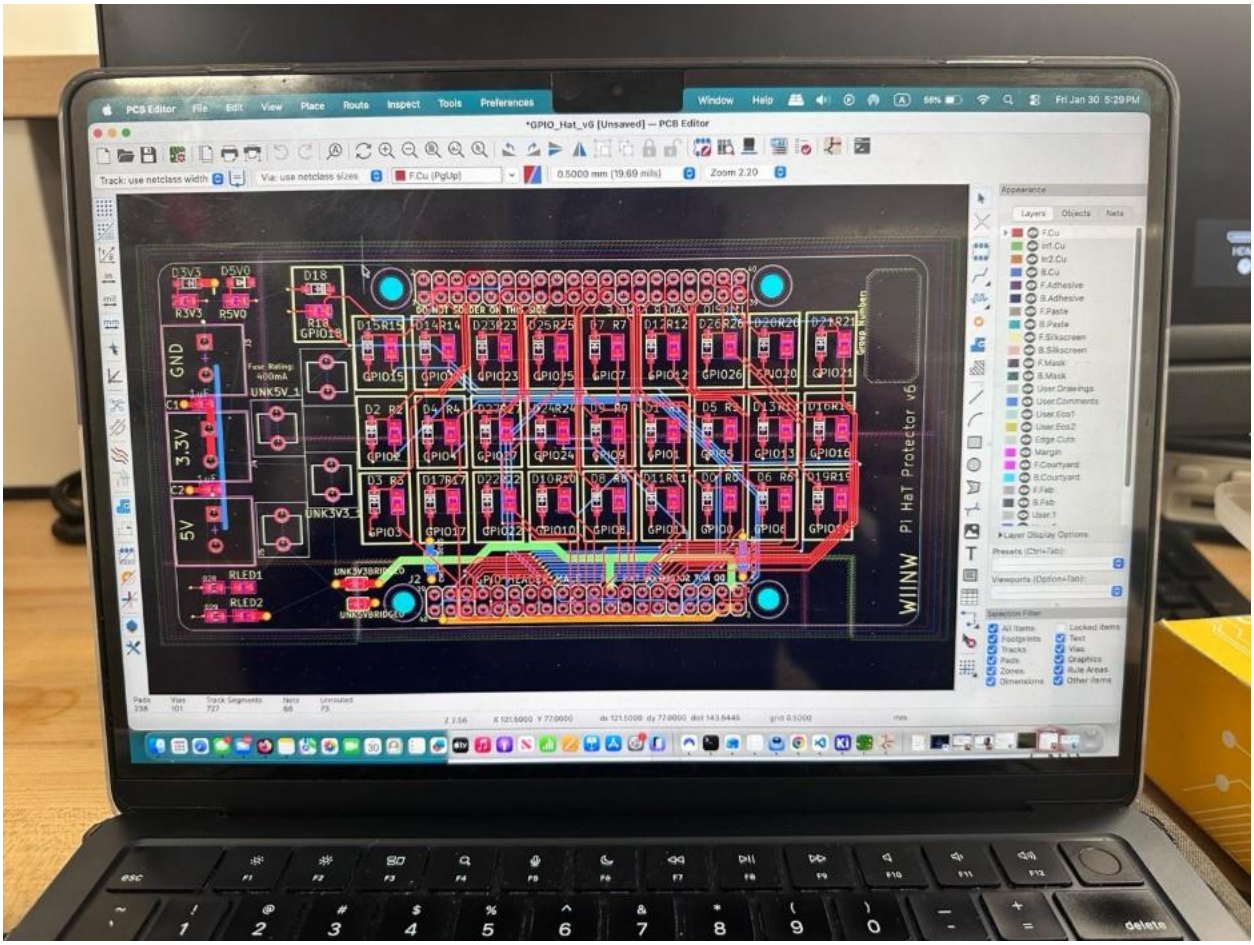

Figure 9. This figure shows the provided PCB opened in the CAD tool for inspection of routing density, layer usage, and protection components. This is important because it gave us a real reference design to learn from before making our own board, especially around the GPIO header where routing gets crowded.

Next, we saved a simplified “blank” board version by stripping away most routing while keeping the GPIO header (and the general board sturcture). This gives us a clean starting point for our own Pi-HAT design later.

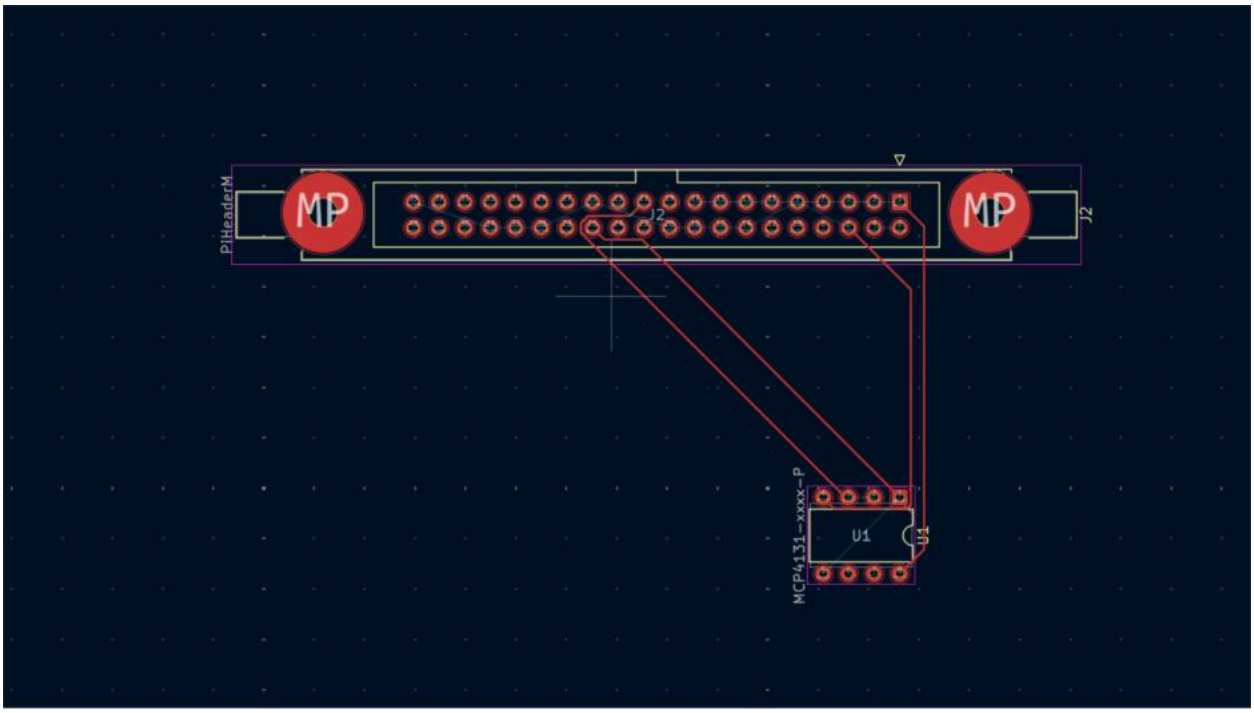

Figure 10. This figure shows a simplified canvas version of the board with the Raspberry Pi header preserved. This is important because it sets up an organized starting point for future PCB development without the clutter of the original routing, while keeping the physical header constraints we must design around.